

# Programación Orientada a Objetos

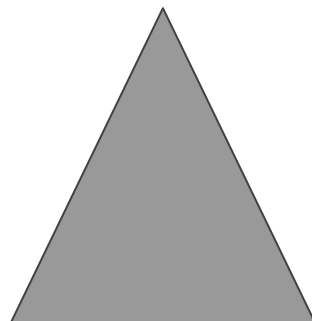
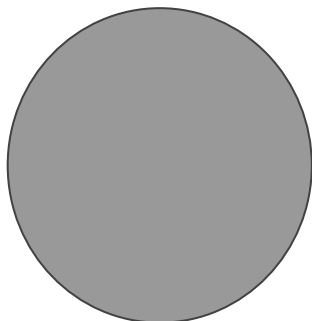


2025

# Figuras

# Formas Básicas

— — —



# Herencia

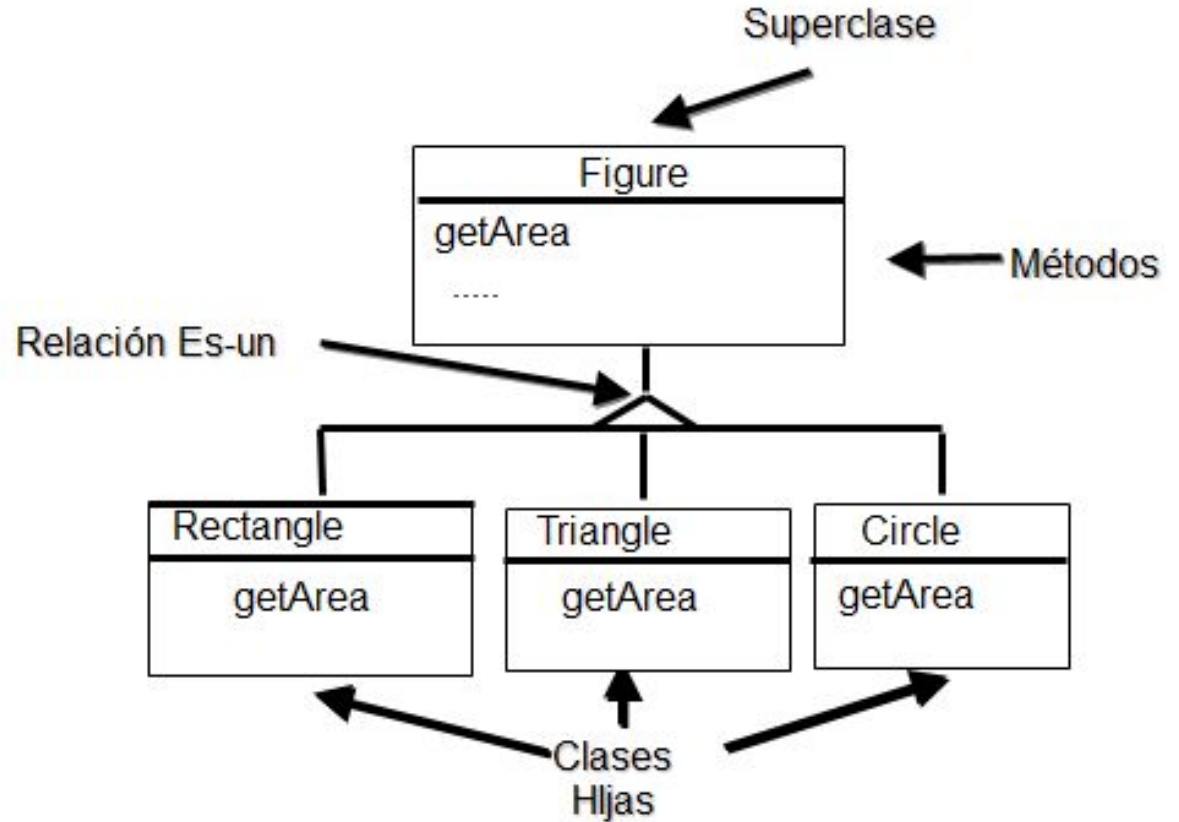
# Herencia

— — —

## **Mecanismo de abstracción, clasificación, extensión y reuso**

- Es posible abstraer características comunes de varias clases en una “superclase”
- EL mecanismo de abstracción sirve como mecanismo de clasificación de entidades
- La extensión permite ampliar las características de una clase en una subclase
- Es un mecanismo de reuso tanto a nivel de diseño como implementación

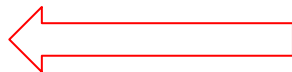
# Herencia: Ejemplo



# Herencia: Ejemplo

---

```
public class Figura {  
    String nombre;  
  
    public double getArea(){  
        return 0.0;  
    }  
  
    public String getNombre(){ return nombre;}  
  
}
```



Existe una simplificación del concepto para introducir la herencia más adelante en el curso vamos a ver el tema de métodos que “no hacen nada”

# Herencia: Ejemplo

---

```
public class Circulo extends Figura{  
    double radio;  
  
    public double getArea(){  
        return Math.PI * radio * radio;  
    }  
}
```

Círculo Hereda de  
Figura

El Círculo es una  
Figura



# Herencia: Ejemplo

---

```
public class Triangulo extends Figura{  
    double base;  
  
    double altura  
  
    public double getArea(){  
        return (base*altura)/2;  
    }  
}
```

Triangulo Hereda de  
Figura

El Triángulo es una  
Figura

# Herencia Ejemplo Constructores

---

Los constructores no se heredan!!!

En la clase Figura:

```
public Figura(String n) {  
    nombre = n;  
}
```

# Herencia Ejemplo Constructores

---

Los constructores no se heredan!!!

En la clase Circulo:

```
public Circulo(int r) {  
    nombre = "Circulo";  
    radio  = r;  
}
```

```
public Circulo(int r) {  
    super("circulo");  
    radio = r;  
}
```

Se puede invocar el constructor de la clase padre (si o si primer línea)

# Ejemplo de Uso de Herencia

---

```
Circulo c1 = new Circulo(4);
```

```
Triangulo t1 = new Triangulo(10,10);
```

```
c1 = t1;      // ERROR “un triángulo no es un círculo”
```

```
t1 = c1 ;    // ERROR “un círculo no es un triángulo”
```

```
Figura ff1 = t1; // SI ! “El triángulo es una figura”
```

```
ff1 = c1; // SI ! “El círculo es una figura”
```

# Ejemplo: Envío de mensajes

---

```
c1.getArea();
```

```
c1.getNombre();
```

```
c1.getRadio();
```

```
ff1 = c1;
```

```
ff1.getRadio(); // ERROR, Java es un lenguaje Tipado
```

# JAVA: Compilación

---

Java controla el envío de los mensajes por el **TIPO** del objeto, es decir el control es estático.

En el ejemplo anterior `ff1` es del tipo `Figura`, y la clase `Figura` no tiene un método `getRadio()`

# super

---

Similar a **this**, la palabra **super** se utiliza para referir al “padre” de la clase. Lo usamos para poder invocar un método y modificar su comportamiento.

Supongamos una clase MedioCirculo (es un círculo pero que tiene la mitad de area)



# super

---

```
public class MedioCirculo extends Circulo{  
    public double getArea(){  
        return super.getArea()/2;  
    }  
}
```

El MedioCirculo es un Circulo cuya área es la mitad del Circulo.  
Por ejemplo si cambio el getArea del Circulo, el MedioCirculo sigue  
cumpliendo la propiedad de que su área es la mitad de la de Circulo



# Herencia

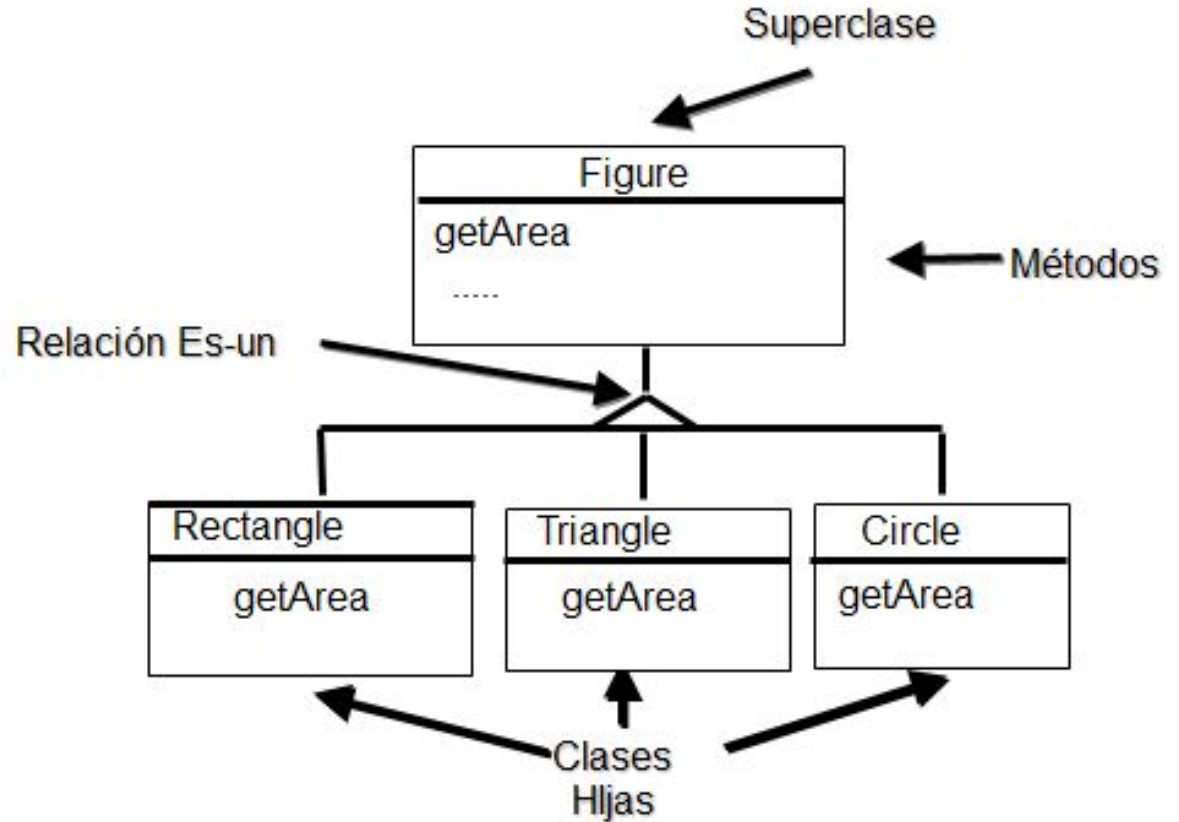
# Herencia

— — —

## **Mecanismo de abstracción, clasificación, extensión y reuso**

- Es posible abstraer características comunes de varias clases en una “superclase”
- EL mecanismo de abstracción sirve como mecanismo de clasificación de entidades
- La extensión permite ampliar las características de una clase en una subclase
- Es un mecanismo de reuso tanto a nivel de diseño como implementación

# Herencia: Ejemplo



# Binding Dinámico

---

Es un mecanismo a través del cual el método que se ejecuta en respuesta a un mensaje se determina dinámicamente dependiendo de la clase a la que pertenezca la instancia que recibió el mensaje.



# Binding Dinámico: Ejemplo

---

```
Figura f4 = new Triangulo(10,10);
```

f4.getArea(); // SI fuera estático, se ejecuta el de la “clase” y no el del Objeto Recién en tiempo de ejecución se sabe el método

# Binding Dinámico

---

Siguiendo el ejemplo

```
if (EL USUARIO APRIETA "1" )
```

```
    f4 = new Circulo(4);
```

```
else
```

```
    f4 = new Triangulo(10,10);
```

```
f4.getArea(); // Que metodo se ejecuta?
```

# Binding Dinámico

— — —

**Recordar que Mensaje era distinto a Método.**

El **mensaje** es la señal que se envía, y el **método** el código que se ejecuta como respuesta a la señal



# Polimorfismo

---

Griego (muchas Formas)

Es la habilidad de una **variable** o **referencia** de tomar valores de diferentes **tipos**, lo que implica la respuesta a los mismos **mensajes**.





# Polimorfismo Ejemplo

---

```
public void imprimirFigura ( Figura ff){  
    System.out.println( “la figura “ + ff.getNombre() +  
        “ tiene un Area de: “ + ff.getArea() );  
}
```

La variable ff puede tomar diversas “formas”, aunque siempre de las que hereden de Figura

**Polimorfismo y  
binding dinámico  
son dos mecanismos  
esenciales que  
permiten el reuso y son  
la base de la potencia y  
elegancia de la POO**



— — —

# Juego de Dados



# Nueva Funcionalidad

---

un jugador es tramposo y utiliza dados cargados que favorecen la salida del **5** y el **6** respectivamente la **mitad** de las veces que se tira el dado

# Dados Cargados

```
public class Dado{  
    int valor;  
    int caras;  
  
    public int tirar() {  
        return (Math.random()*caras)+1;  
    }  
}
```

```
public class DadoCargado5 extends Dado{  
  
    public int tirar() {  
        if (Math.random()>0.5)  
            return super.tirar();  
        else  
            return 5;  
    }  
}
```

```
public class DadoCargado6 extends Dado{  
  
    public int tirar() {  
        if (Math.random()>0.5)  
            return super.tirar();  
        else  
            return 6;  
    }  
}
```



***Mal uso de la herencia: no voy a crear una clase por cada valor de una variable de instancia que cambie***

# Dados Cargados

```
public class Dado{  
    int valor;  
    int caras;  
  
    public int tirar() {  
        return (Math.random()*caras)+1;  
    }  
}
```

```
public class DadoCargado extends Dado{  
  
    int ladoCargado;  
    public int tirar() {  
        if (Math.random()>0.5)  
            return super.tirar();  
        else  
            return ladoCargado;  
    }  
}
```

# Que otra mejora le puedo hacer a la clase Dado

---

Si alguien quiere hacer un dado Cargado con mayor o menor probabilidad que debo hacer con la solución como esta?

Una CONSTANTE en código, siempre deja fija mi solución, ya sea 0.5, “juan”, return 3

Corrigiendo estas cuestiones propiciamos la mejora de nuestra solución

# Dados Cargados

— — —

Que otras clases debo modificar?

Como?



# Clase Vs Instancia

# No puede existir clases iguales

— — —

**Dos clases iguales son la misma**

**CUANDO LA DIFERENCIA ES SOLO UNA CONSTANTE**, entonces también son la misma clase

# Malos ejemplos

— — —

PersonaJuan

PersonaPedro ---> “Juan”

----> Existe un nombre, las dos son personas con una variable Nombre

# Mal en Dados

---

Clase padre Dado

DadoCargado5 y DadoCargado6 ---> Los dos son DadoCargado y el valor en realidad es una **variable!!!**

DESAPARECEN DadoCargado5 y DadoCargado6

# Reconocer fácil el error, cuando es por el uso de Constantes

— — —

```
if (x>150)
```

**NO TIENE QUE HABER  
CONSTANTES EN EL  
CÓDIGO**

```
if (nombre.equals("juan"))
```

```
return "juan"
```

```
return 84;
```

```
return "a";
```

```
return "juan";
```

**reemplazar con:**

```
return nombreADevolver;
```